

AI Enhanced Masked Autoencoder for Detection of Traffic Anomalies

MIDN Blake Powers
Cyber Science Department
United States Naval Academy
Annapolis, United States
m265100@usna.edu

MIDN Tyler Wenzel
Cyber Science Department
United States Naval Academy
Annapolis, United States
m266774@usna.edu

MIDN John Sullivan
Cyber Science Department
United States Naval Academy
Annapolis, United States
m266180@usna.edu

MIDN Matthew Zehnder
Cyber Science Department
United States Naval Academy
Annapolis, United States
m267146@usna.edu

Tony Melaragno, Ph.D.
Cyber Science Department
United States Naval Academy
Annapolis, United States
melarang@usna.edu

Abstract—Data exfiltration is the fastest growing cyber attack and detecting it before sensitive information is grabbed from a system is a critical challenge that public and private entities face. According to the 2025 Microsoft Digital Defense Report, 80% of reactive engagements include data collection [1]. This paper explores the use of masked autoencoders placed at edge devices, in order to provide a layered defense strategy to identify these attacks. We explored the use of autoencoders with two methods, one analyzing packet payloads and the other analyzing traffic flows with metadata. Using a controlled two node environment, we collect generated benign Packet Capture (PCAP) files and use them to train a local autoencoder to differentiate between normal and malicious traffic. Once properly trained, each model continuously receives incoming PCAP files, reconstruct them, and compare them to the normal patterns learned. If traffic does not match these patterns it is flagged as suspicious and the administrator is notified. This provides a simple, lightweight, and effective anomaly detection method. Our results hope to show that this approach will be effective in detecting unusual traffic that can provide a foundation for solving this newly evolving crisis.

Index Terms—autoencoder, entropy, Artificial Intelligence (AI)

I. INTRODUCTION

Each day malicious cyber actors gain access to sensitive systems, hosting troves of personal data, company data, or national security data is no longer the pressing question; the pressing matter is how can it be detected that they are stealing the data. The use of autoencoders to sniff outbound and inbound network traffic is one potential solution explored in our research. To be clear, this is not going to prevent hackers from entering sensitive system unauthorized as this method is only to detect the exfiltration, flag the data being stolen, and then stop further leakage. The use of autoencoders on edge devices is another layer in the defense in depth strategy each company and government entities use to combat the ongoing threat of malicious cyber actors [2]. The main problem with many detection methods is they utilize a lot of resources to include computing power, energy, and memory. With this masked autoencoder design, it is constructed to be light weight

and optimized to work with fewer Arithmetic Logic Unit (ALU)s and restricted memory [3]. Exfiltration attempts look different from regular traffic in aggregate and very large byte counts, long duration, many packets in a short period of time, unusual destination IPs/ports, or protocol mismatches, leading to higher entropy values [4]. Analysis of flows let you detect that without keeping every packet, keeping detection methods lightweight and flexible.

II. RELATED WORKS

Willems et al. [5] researched their implementation of this novel technology to be tested and used at the Dutch National Cyber Security Centre. From their research they found that autoencoders can be implemented on networks to read packet properties to see if the autoencoders can detect malware moving through a network. Autoencoder AI's were trained unsupervised each targeting a specific packet attribute to see if they would be able to detect malware. Their autoencoders were used to encode and then decode data and when trained on data samples creates a legitimate output but when fed a contaminated set the autoencoder will perform worse at encoding it. Essentially it an anomaly checker for packets traveling across the network that checks if aggregation is above threshold. In their research, this worked well in aggregate systems.

Albataineh et al. [6] explores early data exfiltration detection focused on the FTP service using the (Integrated Data Exfiltration Monitoring Tool (IDEMT) tool. IDEMT employs entropy for initial anomaly detection and flags potential exfiltration events for manual review. The tool consists of three components: the Traffic Capture Engine (TCE), the (Traffic Behavior Analyzer (TBA), and the File Access Pattern Analyzer (FAPA). The TCE captures network traffic, which is analyzed by the TBA using correlation coefficients to detect abnormal traffic patterns. FAPA identifies anomalous file access patterns based on normal behavior. While IDEMT can flag potential data exfiltration, it requires manual verification, indicating the

need for further automation or integration with other detection tools for more effective exfiltration detection. This sets up a baseline to how entropy is deployed in anomaly detection.

Mirsky et al. [7] combines the use of Network Intrusion Detection System (NIDS) and neural networks to enhance the capabilities and adaptability of NIDS. Neural networks provide real time learning capabilities that can be adapted for various network architectures and endpoint devices, yet contrasts it with challenge of the need for so many unique neural networks and the vast amount of resources needed to train each neural network. They present a novel NIDS called Kitsune using their *KitNET* algorithm, which can be connected anywhere on a local network where it will learn from the environment and detect anomalies. From there testing, it proved very versatile, being able to be implemented among various devices to even include a Raspberry Pi Pico, proving it's efficiency. Using a similar model to Kitsune, lightweight autoencoders placed throughout a network provide a high surface area, increasing the probability of detection.

Ullah et al. [8] survey various covert data exfiltration techniques, categorizing them into content-based, header-based, and metadata-based methods. Content-based methods include data encoded within Uniform Resource Locator (URL)s of Hyper Text Transfer Protocol (HTTP) 404 requests and image steganography. Header-based methods exploit fields such as the Time-to-Live (TTL) value, where bytes are encoded directly into the TTL field of packet headers. Metadata-based methods encode data using inter-packet timing delays. The paper concludes that existing Intrusion Detection System (IDS) solutions are capable of detecting most exfiltration methods, with the notable exception of timing-based techniques. The authors attribute this limitation to the fact that most IDS implementations analyze individual packet payloads in isolation, creating a detection gap when exfiltration methods exploit traffic flow characteristics rather than packet content. The use of lightweight autoencoders able to analyze traffic flows rather than just the packet payloads would potentially add another layer of defense in a defense in depth strategy.

III. NETWORK ENVIRONMENT

The testing environment consisted of two virtual machines running on in a Proxmox environment and connected via a Linux Bridge. These instances will be referenced as node one, and node two. Each node has 8 cores running x86-64-v2-AES, 16 GB of memory, and are running Ubuntu 24.04. As shown in table I, node one is used as the collection node while node two is the command and control node for the simulated malicious implant on node one.

Node ID	IP Address	Purpose
Node 1	192.168.197.93	Client Node
Node 2	192.168.197.20	Command and Control Node

TABLE I
NETWORK NODE ASSIGNMENTS

The data that was used to train the autoencoder was collected from node one. This host was running two different traffic generation scripts to simulate both benign user interaction with the host, alongside malicious traffic. This method enables the use of a lightweight and personalized autoencoder trained on data that originates only from the node on which it is utilized. This approach legitimizes the approach of using small scale AI for threat detection unique to a single station. This concept can be expanded to a greater network over more stations or consolidated to more specific PCAPs revolving around a singular protocol.

A. Benign Traffic Generation

To collect benign traffic representative of a system administrator for training the autoencoder, a purpose-built user traffic emulation script was deployed on Node 1. Rather than producing synthetic or uniform packet streams, the script replicates the behavior patterns typical of a system administrator within a corporate environment. Uniform scripted traffic would produce artificially low variance in the produced PCAP files, causing the autoencoder to train with an unrealistically narrow baseline and increasing false positives when exposed to real-world traffic.

The script uses a probabilistic state machine that selects one of three behavioral states during working hours. With a probability of 0.60, the script enters a Research state, performing multi-request web browsing sessions against a pool of external sites using `curl` with rotating User-Agent strings representative of common enterprise browsers. With a probability of 0.30, the script enters a Work state, establishing a Secure Shell (SSH) session with node two and executing three to six commands with randomized delays between each to simulate operator read time. The if either of these states are not selected, the script triggers an idle state lasting between 60 and 30 seconds, representing brief periods of inactivity. A schedule constraint limits traffic generation to a 0900 to 1700 window on weekdays, ensuring the baseline dataset reflects realistic working hours. Inter-event timing is modeled using Gaussian distributions to produce the natural jitter observed in human sessions, and burst behavior is explicitly encoded in the Research state through multiple sequential requests and probabilistic background Domain Name System (DNS) lookups to simulate browser pre-fetching. Together, these mechanisms ensure that the PCAP files captured by `tshark` reflect the statistical properties of genuine employee traffic, providing the autoencoder with a training distribution that generalizes to live network conditions.

State	Probability	Simulated User Action
Research	0.6	GET requests, DNS Lookups
Work	0.3	SSH session with node two
Idle	0.1	None

TABLE II
THE VARIOUS USER STATES SIMULATED BY THE BENIGN TRAFFIC GENERATOR.

B. Malicious Traffic Generation

To simulate an active data exfiltration event, a two-component implant was deployed concurrently alongside the benign traffic emulator on node one. The client runs on node one and periodically transmits randomly generated payloads to a Flask-based command and control script on node two. When running alongside the benign traffic generator, this implant simulates a mixed-traffic environment in which the autoencoder must distinguish normal behavior from an actively beaconing implant.

The client operates on a fixed 300-second transmission window subdivided into four sub-bursts, flushing data approximately every 75 seconds. Payloads are partitioned into chunks and sent sequentially to the server's endpoint route over HTTP using headers that mimic a standard browser environment, blending the malicious traffic with the benign browsing activity already present on the network. This obfuscation simulates techniques common in real-world implants, enhancing the real-world applications of our autoencoder. On node two, the server accepts each JavaScript Object Notation (JSON) POST request, logs the payload with a timestamp and character count, and returns a success acknowledgment to the client. This exchange repeats four times per window, producing a regular, low-volume outbound stream from node one to a fixed internal destination. This structured, periodic pattern is precisely the kind of exfiltration behavior the autoencoder is trained to detect.

IV. THE AUTOENCODER

The autoencoder in our system is used to learn what normal network traffic looks like so that it can spot anything that stands out as unusual. Instead of relying on labeled attack data, which is hard to obtain, the autoencoder is trained only on benign PCAP files collected from node eight using tshark before any malware is introduced. This matches the same core idea used in the Kitsune [7] An Ensemble of Autoencoders for Online Network Intrusion Detection paper where the model first learns the regular patterns found in network flows such as how many bytes are normally sent, how long connections last, packet rates, and which destinations a host talks to. During training the autoencoder compresses these features, reconstructs them, and gradually learns what the normal behavior of our test network looks like. Once training is complete, any traffic that does not reconstruct the way normal data does is treated as suspicious because the autoencoder recognizes that it does not fit the patterns it learned.

To better detect data exfiltration, we will also incorporate sliding window flow summaries that give the autoencoder a fuller view of each host's recent behavior. This is because a single flow on its own may not provide enough information, but when multiple flows are grouped together the autoencoder can see larger changes such as sudden bursts of traffic, unusual destination addresses, or spikes in outbound data [7]. These aggregated features are then fed into the autoencoder as shown in Figure 1, where traffic is encoded, decoded, and compared

back to the original. The entire process runs automatically as new PCAP files are collected, allowing the model to check live traffic against the threshold it learned during the initial forty eight hour training period. This makes the autoencoder function as a simple but effective way to flag traffic that looks out of place without needing signatures or predefined rules. Again, Figure 1 shows the way the data is fed to the autoencoder, to then be encoded and decoded (or reconstructed as shown in Figure1. The encoder maps the input data to a lower-dimensional latent representation at the bottleneck layer, capturing the most salient features of the normal traffic distribution. The decoder then attempts to reconstruct the original input from this compressed representation, and the reconstruction error is measured. This error value is then used to determine whether a given sample deviates from the learned normal baseline [9]. The benign data collected by tshark on node 8 is saved as PCAP files, all completed before malware is introduced. These PCAP files are then what are fed into the autoencoder. This process is done in real time as PCAP files are saved and then sent straight to the autoencoder for encoding and decoding. The autoencoder is trained unsupervised for a period of 48 hours, where it develops it's own threshold to compare the reconstructed data to the original. We implemented the autoencoder in two main ways, one that analyzed individual packet payloads, and another that analyzed packet flows.

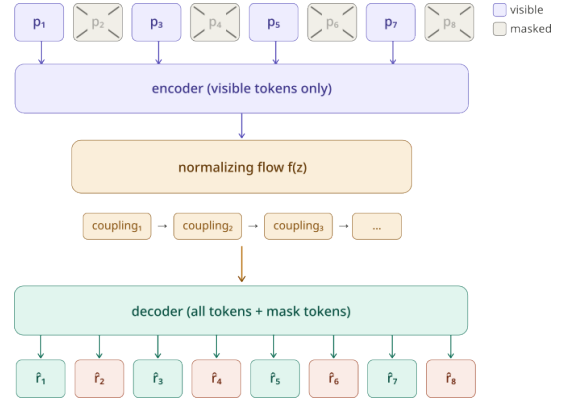


Fig. 1. Masked autoencoder; A single autoencoder conducting training using benign data to create an acceptable error threshold.

A. Auto-encoding of Individual Packets

The first iteration of the proposed model employed individual packet autoencoding, in which each packet was associated with a five-tuple consisting of the source IP address, source port, destination IP address, destination port, and protocol. Each packet was independently processed by the autoencoder. Each was encoded, decoded, and subsequently evaluated against a predetermined threshold value. This approach was hypothesized to yield more granular and precise analysis at the individual packet level.

B. Auto-encoding of Packet Flows

Packet flow analysis is the second iteration of the auto-encoder. This would follow Transmission Control Protocol (TCP) handshakes, Hypertext Transfer Protocol Secure (HTTPS) packets, and SSH connections to analyze the entire flow of data. As then the entropy of each flow could then be compared to a threshold value, giving each flow a score itself to then whether an entire flow itself. We hypothesized that this approach would yield improved results, as individual HTTPS packets exhibit elevated entropy due to their encrypted payloads, causing them to be disproportionately flagged as suspicious when analyzed in isolation. This limitation extends beyond HTTPS, as each network protocol carries a distinct baseline entropy level [10], meaning that packet-level analysis alone may be insufficient for reliable anomaly detection across mixed-protocol traffic. This we suspect would limit our ability to detect exfiltration attempts using protocols that do not have as long of a stream such as User Datagram Protocol (UDP) connections.

V. COLLECTION AND PROCESSING

The collection and processing part of the data is important to transform raw data into something actionable and readable by the autoencoder. Tshark is the main tool used in our collection. As for processing, that is our autoencoder assigning traffic flows risk levels to determine whether the flow may be exfiltration or not. Figure 2 shows the process followed for

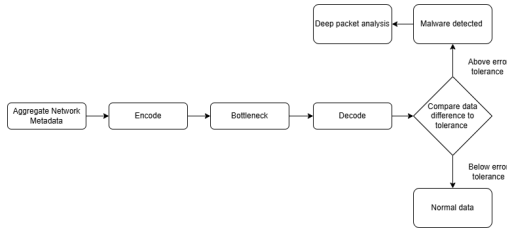


Fig. 2. Training Flow Chart

each piece of flow tested against our autoencoder. Collection and aggregation of traffic data was done by node one, utilizing tshark in order to collect the network traffic on the loop back address and save it as a PCAP file. Clean traffic (no beaconing to node two) was the first set of data generated by our script. The script was ran for a total of 48 hours on node one. Once complete with collection, the PCAP and Transport Layer Security (TLS) keys were both saved as they were ready for the autoencoder.

These PCAP files is then fed into the autoencoder also housed in node two, where the PCAP data is encoded. The data is then reconstructed by the autoencoder. This reconstructed data is then fed into a comparison model trained on benign data. The comparison model has a developed threshold that is used during the comparison phase between the reconstructed and initial data.

VI. RESULTS

The Masked Flow Autoencoder was tested against 2 primary sets of data. The first set of data contains both benign and malicious packet data. The results of which can be seen in Figure 3. Showing a high number of packets caught with large reconstruction errors. The other data set that was tested on was a PCAP of purely benign data to ensure the Autoencoder did not have too many false positives. The results of which can be seen in Figure 4 showing a much lesser amount of packets with abnormally high reconstruction errors.

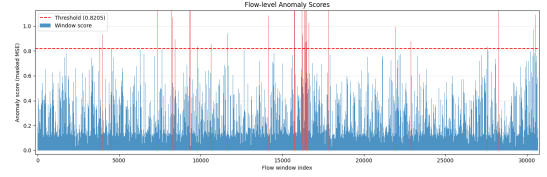


Fig. 3. The results of the Autoencoder after run against a PCAP with malicious data, resulting in higher anomaly scores.

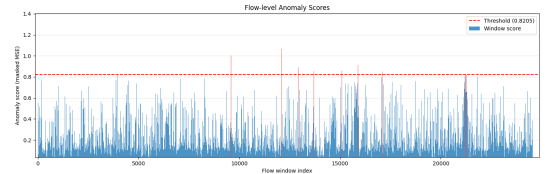


Fig. 4. The results of the Autoencoder when ran against a PCAP with benign data, resulting in lower anomaly scores.

A. Training

The initial creation and training of the model was done on 48 hours of packet data. Creating the model took approximately 45 minutes to parse through over 2GB of data, train on the data, and create the model. After initial creation further training was much quicker. To add 4 gigabytes worth of packets for further training only took 10 minutes to complete. The system used to create and train the models includes a 3080 ti, AMD Ryzen 9 5900X 12-core processor, and 32 gigabytes of ram. Training the AI in this way helps to prove a few different concepts the autoencoder approaches. With the speed of creation and training provides a opportunity for a light weight personalized malware detection system. Giving an option for independent people or smaller companies to have custom malware detection. By using only minimal data to train the AI helps to show viability even in early stages of implementation.

B. Testing

Data which the autoencoder was tested on was 48 hours of normal traffic, being ran along side malicious data is the shape of a key logger. When testing the program against the data the largest time usage was not the Autoencoder training, but parsing the packets into the flows and necessary 5 tuples to for the autoencoder to test against. The evaluation metrics of the

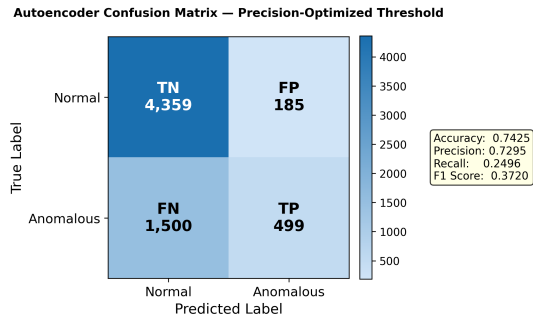


Fig. 5. A confusion matrix of the autoencoder run against a PCAP with malicious data.

Autoencoder were an Accuracy of .7425, Precision of .7295, Recall of 0.2496, and F1 score of 0.3720 as seen in Figure 5. This score was achieved after optimizing the threshold for Precision over Recall. The Accuracy in this case tends to be miss leading do to the much higher number of good data within the test, as if nothing was flagged the Accuracy would still be at roughly 69%. A Precision 0.7295 bods good results meaning that approximately 72% of the time an item flagged it was malicious. This precision was surprising for an autoencoder trained only on seven days worth of benign data. Recall is the biggest problem that we came across while testing the autoencoder. With a value of 0.2469 over three quarters of the malicious data made it past the autoencoder without detection. This deficiency in Recall resulted in a F1 value of 0.3720. While the F1 and Recall value are low the higher Precision resulting in few false flags is worth the trade off for the purpose of the autoencoder. One way to increase the autoencoders Recall was more training. The earlier model trained off of just two days worth of data had a Precision and Recall value at 0.6698 and 0.2174, proving that training the model on more benign data results in an improved autoencoder.

C. Viability

Due to increased stats after further autoencoder training there is viability to have this be implemented as a long term malicious detector. By sitting on a node for extended periods the autoencoder can learn from all the traffic coming in gaining a higher F1 value as time goes by. Under the current set up while much of the malicious traffic makes it through patterns can still be seen within the results that could allow the malicious source to be caught and brought down. When analyzing the results of the malicious data 9 out of the 10 highest error scores belong to the malicious data and are going to the same place. This could allow for discovery of the malicious actor even when most of the bad flow went by undetected.

VII. CONCLUSION

The results demonstrate that a Masked Flow Autoencoder can serve as a viable lightweight solution for personalized network anomaly detection. By training exclusively on benign traffic, the model learned a reliable baseline of nor-

mal behavior, allowing it to flag deviations with reasonable precision while minimizing false positives. Critically, the experiments confirmed that continued training on benign data progressively improves model performance, suggesting that the system grows more effective the longer it is deployed in a real environment. The efficiency of both initial training and incremental updates further supports the feasibility of this approach for individuals and smaller organizations without access to enterprise-scale security infrastructure. Future work should focus on improving recall through expanded training datasets, exploring additional traffic feature representations, and evaluating the model against a broader range of attack types.

VIII. FUTURE WORK

The use of a light-weight autoencoder has a promising future. Its implementation is not limited to malicious data exfiltration, but extends to any network anomaly detection. A future case we see as possible is using an autoencoder to detect insider threats. For example, an employee's workflow would be monitored and a baseline would be established. The autoencoder, much like its implementation for data exfiltration, would then alert a security officer when an anomaly is detected in their workflow, allowing them to dive deeper into network logs and file transfer patterns for a closer look. The goal would be to reduce the scope of information that security officers and counterintelligence personnel have to review when trying to detect insider threats. This autoencoder software detects anomalies in an adaptive manner, allowing it to be deployed without the need for constant updates. It is also a light-weight implementation that does not require a large CPU or significant memory to execute, meaning it could be run on something as small as a 4GB Raspberry Pi 4. This would allow it to be used by resource-constrained public and private networks. By training the model on a network's baseline traffic, the system learns to detect anomalous behavior within that specific environment without requiring labeled attack data or signature updates. Traditional signature-based tools like Snort and Suricata rely on predefined rule sets and known attack patterns, making them blind to zero-day threats that have not been previously recorded. This lightweight autoencoder-based network intrusion detection system provides continuous, unsupervised anomaly detection at a fraction of the cost of commercial NIDS platforms, giving smaller enterprises and government organizations an accessible alternative to protect their networks. The practical and scalable nature of this technology makes it well-suited for securing networks that currently operate without dedicated threat detection.

ACKNOWLEDGMENTS

We would like to extend our gratitude to Dr. Anthony Melaragno, Ph.D. for his help providing the network environment and nodes for us to conduct our research on and his continued support throughout the entire capstone process.

DISCLOSURE

The views expressed in this paper are those of the authors and do not reflect the official policy or position of the U.S. Naval Academy, Department of the Navy, the Department of Defense, or the U.S. Government.

REFERENCES

- [1] Microsoft, “Microsoft digital defense report 2025,” Microsoft Corporation, Annual Report, 2025, accessed: April 2, 2026. [Online]. Available: <https://www.microsoft.com/en-us/security/security-insider/microsoft-digital-defense-report-2025>
- [2] National Institute of Standards and Technology, “Security and privacy controls for information systems and organizations,” National Institute of Standards and Technology, Gaithersburg, MD, Special Publication NIST SP 800-53 Rev. 5, 2020. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-53r5>
- [3] B. S. Sharmila and R. Nagapadma, “Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset,” *Cybersecurity*, vol. 6, no. 1, p. 41, 2023. [Online]. Available: <https://doi.org/10.1186/s42400-023-00178-5>
- [4] P. Dorfinger, G. Panholzer, and W. John, “Entropy estimation for real-time encrypted traffic identification,” in *Proceedings of the International Workshop on Traffic Monitoring and Analysis (TMA)*. Springer, 2011. [Online]. Available: https://www.sjalander.com/wolfgang/publications/TMA11_dorfinger.pdf
- [5] D. Willems, K. Kohls, B. van der Kamp, and H. Vranken, “Data exfiltration detection on network metadata with autoencoders,” *Electronics*, vol. 12, no. 2584, pp. 1–20, 2023.
- [6] A. Al-Bataineh and G. White, “Analysis and detection of malicious data exfiltration in web traffic,” in *Proceedings of [Conference/Workshop Name]*, San Antonio, TX, 2012, emails: aal-bata@cs.utsa.edu, greg.white@utsa.edu.
- [7] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” 2018. [Online]. Available: <https://arxiv.org/abs/1802.09089>
- [8] F. Ullah, M. Edwards, R. Ramdhany, R. Chitchyan, M. A. Babar, and A. Rashid, “Data exfiltration: A review of external attack vectors and countermeasures,” *Journal of Network and Computer Applications*, vol. 101, pp. 18–54, 2018.
- [9] K. A. Alaghbari, H.-S. Lim, M. H. M. Saad, and Y. S. Yong, “Deep autoencoder-based integrated model for anomaly detection and efficient feature extraction in IoT networks,” *IoT*, vol. 4, no. 3, pp. 345–365, 2023. [Online]. Available: <https://www.mdpi.com/2624-831X/4/3/16>
- [10] A. Kenyon, D. A. Elizondo *et al.*, “Characterising payload entropy in packet flows: Baseline entropy analysis for network anomaly detection,” *Future Internet*, vol. 16, no. 12, p. 470, 2024. [Online]. Available: <https://www.mdpi.com/1999-5903/16/12/470>